

P1406R1

Add more `std::hash` specializations

New Proposal, 2019-04-24

Authors:

[Alexander Zaitsev](#) (Solarwinds) zamazan4ik@tut.by

[Antony Polukhin](#) (Yandex Taxi) antoshkka@gmail.com

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LEWGI, LEWG, LWG

Source:

https://github.com/ZaMaZaN4iK/ConfsANDProps/blob/master/Proposals/P1406_complex_hash/complex_hash.bs

Abstract

In Standard library we already have `std::hash` specializations for some classes like `std::string`. Unfortunately, we have no specializations for a lot of other classes from Standard Library like `std::array`, `std::tuple`, etc. At the moment people who need hash calculations for such containers must use `Boost.Hash` functions or write `std::hash` specialization manually. This proposal adds `std::hash` specializations for different containers from Standard Library. Addresses an issue LWG #1025.

Table of Contents

- 1 Changes since R0
- 2 Design decisions
- 3 Proposed wording
- 4 Possible implementation
- 5 References

§ 1. Changes since R0

- Removed a requirement that different containers with the same content should return the same hash value.
- Left `std::hash` specializations only for `std::tuple`, `std::pair`, `std::array`, `std::basic_string`. Other specializations are moved to another paper because of lack of motivation.

§ 2. Design decisions

- We do not require same hash values for different containers that are not comparable (for example `std::pair<int, int>`, `std::tuple<int, int>`, `std::array<int, 2>`).

§ 3. Proposed wording

Add a new Section "19.4.6, Hash support [pair.hash]", with following content:

```
template<typename A, typename B>
    struct hash<pair<A, B>>;
```

Enabled if specializations `hash<remove_const_t<A>>` and `hash<remove_const_t<B>>` are both enabled, and disabled otherwise.

Add a new Section "19.5.3.11, Hash support [tuple.hash]", with following content:

```
template<typename... T>
    struct hash<tuple<T...>>;
```

Enabled if specialization `hash<remove_const_t<U>>` is enabled for every template argument `U` in the parameter pack, and disabled otherwise.

Add a new Section "21.3.7.7, Hash support [array.hash]", with following content:

```
template<typename T, std::size_t N>
    struct hash<array<T, N>>;
```

Enabled if specialization `hash<remove_const_t<T>>` is enabled, and disabled otherwise.

Remove a paragraph from Section "20.3.5, Hash support [basic.string.hash]", with following content:

```
template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u8string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;
```

Add a new paragraph to Section "20.3.5, Hash support [basic.string.hash]", with following content:

```
template<typename charT, typename Allocator>
    struct hash<basic_string<charT, char_traits<charT>, Allocator>>;
```

Enabled if specialization `hash<remove_const_t<charT>>` is enabled, and disabled otherwise.

§ 4. Possible implementation

Some possible implementations can be found in [Boost.Hash](#) library.

§ 5. References

[Boost.Hash](#)